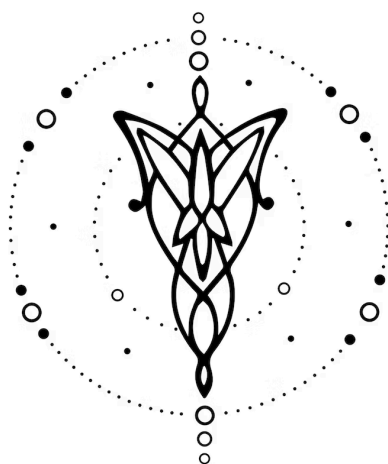




Primer Entrega

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Lola	Diaz Varela	62792	ldiazvarela@itba.edu.ar
Federico	Magri	64508	fmagri@itba.edu.ar
Manuel	Cortés Teyssier	64159	mcortesteyssier@itba.edu.ar
Sebastian	Vega Garrot	64111	svegagarrot@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Proyecto TLA](#)

3. Dominio

MakeLite-C es un lenguaje diseñado para describir la construcción de proyectos en C de manera más clara y legible que los Makefiles tradicionales. Su propósito es simplificar las tareas habituales de compilación al permitir declarar de forma directa los archivos fuente, los directorios de inclusión, los flags de compilación, las bibliotecas y el ejecutable final.

El lenguaje está pensado para proyectos pequeños y medianos, ofreciendo una sintaxis declarativa que reduce la complejidad y evita errores comunes asociados al formato clásico de Make, como problemas de tabulación o dependencias mal definidas. De este modo, MakeLite-C busca facilitar la lectura, la escritura y el mantenimiento de la configuración de un proyecto en C, manteniendo un estilo más cercano a la lógica del propio programador que al del sistema de construcción.

4. Construcciones

El lenguaje ofrecerá las siguientes construcciones y funcionalidades:

- (I). Proyecto:
 - (I.1). Definir el nombre del proyecto y el binario de salida.
- (II). Fuentes y headers:
 - (II.1). Listar archivos fuente (.c) y directorios de includes.
- (III). Flags de compilación y enlazado
 - (III.1). flags: para CFLAGS.
 - (III.2). libs: para bibliotecas dinámicas o estáticas.
- (IV). Estructura de carpetas
 - (IV.1). Definir carpetas de src/, bin/, obj/ (con defaults).
- (V). Reglas automáticas
 - (V.1). Generación implícita de .o desde .c.
 - (V.2). Inclusión de reglas clean, run, test.
 - (V.3). Ejecución de comandos previos y/o posteriores a la compilación.

(VI). Targets personalizados

(VI.1). Permitir definir targets adicionales.

(VII). Macros y variables

(VII.1). Variables que luego pueden interpolarse.

(VIII). Compatibilidad mínima

(VIII.1). Solo se soportan proyectos escritos en C (no C++).

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que compile un proyecto mínimo con un solo archivo fuente.
- (II). Un programa que compile un proyecto con múltiples fuentes y flags de compilación.
- (III). Un programa que use variables definidas por el usuario en las secciones de configuración.
- (IV). Un programa que enlace librerías externas al proyecto.
- (V). Un programa que genere un ejecutable con nombre distinto al del proyecto.
- (VI). Un programa que ejecute comandos previos y posteriores a la compilación.
- (VII). Un programa que compile correctamente aunque no se definan headers.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa MakeLite-C malformado en su sintaxis.
- (II). Un programa que no incluya ningún archivo fuente.
- (III). Un programa que declare un archivo fuente inexistente.
- (IV). Un programa que utilice una variable no declarada.
- (V). Un programa que declare una variable con un nombre reservado.
- (VI). Un programa que defina la misma librería dos veces.

6. Ejemplos

6.1: Creación de una calculadora con tres archivos fuente y un header. Se ejecuta con los argumentos 5 y 10.

```
project calculadora
compiler gcc
flags {-Wall -O2}
```

```
src {main.c, suma.c, rest.c}

headers {calculadora.h}

build

run 5 10
```

6.2: Declara un proyecto llamado helloWorld con un único archivo fuente utilizando gcc como compilador por default y ejecuta ./helloWorld.

```
project helloWorld
src { main.c }
build

run
```

6.3: El proyecto utils se compila con clang, usando los flags -Wall -g aprovechando el uso de variables y linkeando con la librería matemática -lm.

```
var OPTS = -Wall -g
var FILES = main.c util.c

project utils

compiler clang
flags {use OPTS}
src { use FILES }
libs { m }
build
```

6.4: Compila el ejecutable suma y define dos casos de prueba automáticos, verificando que la salida del programa sea la esperada.

```
project suma

src {main.c suma.c}
headers {suma.h}

build
```

```
test {  
  
run 2 3 => 5  
run -1 4 => 3  
  
}
```

6.5: Genera un ejecutable de nombre *custom_bin* distinto al programa fuente de nombre *my_project*

```
project my_project  
  
compiler gcc  
flags {-O2}  
src {entry.c helper.c}  
  
output custom_bin  
  
build  
run arg1 arg2
```

6.6: Ejecuta comandos previos y posteriores a la compilación con los comandos *pre_build* y *post_build* respectivamente.

```
project build_automation  
  
compiler clang  
flags {-Wall}  
  
src {main.c}  
  
pre_build {  
    rm -rf build/  
    mkdir build  
}  
  
post_build {  
    cp output_bin /usr/local/bin/  
    echo "Build completado" > log.txt  
}
```

```
build
```

```
run
```

6.7: Usa variables para flags reutilizables, enlaza múltiples librerías (math y pthread), genera objetos implícitamente de fuentes .c, incluye reglas automáticas como clean (para limpiar artefactos de build), run y test, y define un target personalizado para generar documentación con Doxygen:

```
var OPTS = -Wall -g
var LIBS = m, pthread

project advanced_utils

compiler gcc
flags {use OPTS -O2}

src {main.c, thread_ops.c, math_utils.c}

libs {use LIBS}

clean { rm -rf *.o build/ bin/}
run ./advanced_utils test_arg

test {

run 1 2 => 3
run -5 10 => 5

}

target docs {
    doxygen Doxyfile
}

build
```